ENGINEERING · CONSENT-FIRST MEASUREMENT

JS Execution Order — Challenge & Solution

The client-JS execution-order contract: consent before measurement, idempotent loaders, config-driven tenant pinning, capture-phase form interception — and the tests that enforce it.

Status: **Live · Class: Engineering challenge record · Surface: every page that loads the CRM Sync embeds**

The client-side script stack on a CRM Sync–connected site has one governing rule: **consent state must be fully resolved before any measurement script loads, and every loader must be idempotent across generations.** This document records the challenges that rule answers, the solution architecture, and the automated tests that keep it true.

The Challenge

A CRM Sync page composes up to four script layers — a head loader ("helmet"), a footer loader, the fetched footer embed, and per-page module scripts. Five failure modes emerged, each real, each observed or provoked during development:

1. Measurement before consent. If GA4's `gtag.js` executes before a Consent Mode v2 default is set, the first pageview fires unconsented — a GDPR/ePrivacy violation that no later consent update can retract.

2. Granted visitors measured as denied. The inverse failure, silent and costly: the consent banner recorded the visitor's grant in storage but only *fired* the

consent signal on a **new** save. A returning, consented visitor sat at denied-default for their entire session. Compliant — but the granted signal a merchant is entitled to was discarded, every visit, invisibly.

3. Double-loading loaders. Two loader generations exist across sites (a canonical footer loader and an older inline loader still pasted on legacy pages). Each guarded itself with a different flag, so the two could not see each other. A footer module carrying the canonical tag onto a legacy page executed the embed twice: two consent banners, two form interceptors — **two submissions per form submit**.

4. Tenant misrouting. Environment data (which Shopify store a site pairs with) hardcoded into script tags and element attributes meant that promoting a site from a staging store to a production store required editing markup across pages and modules — and a missed edit routed live submissions to the wrong (locked) store.

5. Platform double-handling of forms. A form submission handled by both the site platform's native handler and the CRM socket produces divergent state: the platform records a submission the data plane never saw, or vice versa.

The Solution

Layer order (document order, enforced)

```
1. stack-loader  (site head, FIRST script tag — the helmet)
2. footer-loader (site-wide, idempotent)
3. footer embed  (fetched + executed by the footer loader)
4. page modules  (docs table, KB search, page snippets)
```

Deferred scripts execute in document order, so ordering is a *markup contract*, not a timing hope.

Consent fires first — the four-step sequence

The helmet emits, in strict source order, all synchronous before any network fetch:

```
1. idempotence guard                       (before any side effect)
2. consent DEFAULT — everything denied     (Consent Mode v2, wait_for_update)
3. stored-consent REPLAY                   (the visitor's saved choice, as an UPDATE)
4. GA4 injection                           (only now, and only with a public config id)
```

Step 3 is the fix for Challenge 2: the visitor's stored decision replays on **every** load, between the denied default and the measurement script. A returning granted visitor is granted from their first pageview; a rejected visitor stays denied; a new visitor is denied until the banner records a choice. Live banner changes flow through the same update channel afterward.

Idempotence across loader generations

The canonical loader respects **and sets** both its own guard flag and the legacy generation's flag. Either loader landing on a page where the other already ran is a no-op. New loader generations must join the existing guard set — a new flag alone recreates Challenge 3.

Tenant pin as configuration, with a precedence ladder

Which store a site pairs with is **configuration, not markup**:

```
element attribute  (one-off override)
→ loader attribute (page-level manual pin)
→ origin→shop pair (server config: requesting site resolved to its store)
→ platform default
```

The server resolves the requesting origin against a config map and serves the embed pre-pinned; the client-side precedence is preserved by an `||` assignment. Promoting a site between stores is one configuration write — zero markup edits, zero republishes (Challenge 4).

Capture-phase form interception

Socketed forms are intercepted at the document level in the **capture phase**, and the handler stops propagation *before* its network call. The platform's own submit handler never receives the event; native browser validation (required, type, pattern) still runs first because it precedes the submit event itself (Challenge 5).

Transition-only escalation

Downstream consent escalation (advertising-signal suppression, audience membership, native email-marketing unsubscribe) fires only when consent state actually *transitions*. Re-affirmations are recorded in the audit trail but never re-fire the escalation — no signal churn, no audit noise.

Verification — the test harness

The contract is enforced by a static suite in the platform TDD harness that asserts the *order of the emitted code itself*: substring-order assertions over the

worker source, so a reordering refactor goes red in CI before a browser ever misbehaves.

Run it after **any** change to the helmet, the loaders, the embed, or a consent path:

```
npx tsx tests/harness/runner.ts --suite=js-load-order
```

ID	INVARIANT	SEVERITY
ORD-G01	Consent default → stored replay → GA4, in emitted order	critical
ORD-G02	Stored replay is a consent UPDATE, never a second default	critical
ORD-G03	Consent default precedes GA4 injection	critical
ORD-G04	Footer loader respects AND sets both generation guards	critical
ORD-G05	Helmet idempotence guard precedes all side effects	high
ORD-G06	Loader sets the tenant pin before fetching the embed	high
ORD-G07	Served embed prepends the server pin; client attribute wins	high
ORD-G08	Form tenant cascade: element attribute → page pin	critical
ORD-G09	Form submit listener binds in capture phase	critical
ORD-G10	Prevent + stop propagation precede the socket's network call	critical
ORD-G11	Consent escalation fires on transitions only (both call sites)	critical
ORD-G12	Anonymous consent parks locally; server consent is token-bound	high
ORD-G13	No backslash regex literals in template-emitted embed code	high

The suite ships beside a companion skill that documents each invariant for the next engineer (or agent) who touches the stack — the two artifacts cross-reference, so the contract cannot silently drift from its enforcement.

Loop remediation checklist

When the suite goes red — or a symptom below appears live — run the loop. Do not skip steps: each one exists because skipping it produced a real incident.

1. **Detect & classify.** Match the red ORD id (or the live symptom via the map below) to its invariant. One id = one invariant = one remediation.
2. **Fix the order, not the test.** The suite asserts the contract; if it disagrees with the code, the code moved. Restore the emitted order or guard. The only time the test changes is when the *contract* changes deliberately — and then this document changes in the same commit.
3. **Re-run the suite locally** until all thirteen are green.
4. **Deploy the worker** — from the worker directory, with its own config file. A deploy launched from the wrong directory fails silently behind pipes; confirm a fresh Version ID in the output before proceeding.
5. **Wait out edge-cache convergence.** Served loaders cache at the edge for five minutes. Verify every edge copy serves one hash before trusting a live check — a mixed edge serves old and new code simultaneously (and if a subresource hash is registered on the site, a stale copy fails integrity and loads nothing).
6. **Verify live, both halves.** Fetch the served embed for the target origin and confirm the pin/markers; then a real browser submit on the published page — success state shown, exactly one request, correct `?shop=` on the socket call.
7. **Record.** Commit with the invariant named; the reconcile/audit trail should already show the corrected behavior on the next real event.
8. **If the change added a new loader generation, consent writer, or interceptor:** extend the guard set / transition guards, add the matching ORD assertion,

and update this document — in the same change, not a follow-up.

Symptom map

SYMPTOM	VIOLATED INVARIANT
Two consent banners; duplicate form submissions	Loader idempotence guards
Returning consented visitor measured as denied	Stored-consent replay
Events fire before any consent decision	Default-before-measurement; helmet not first in head
Submission routed to the wrong store	Tenant pin cascade / missing origin pair
Platform success state *and* socket submission both fire	Capture-phase + stop ordering
Audience membership churns on every banner save	Transition-only escalation

CRM Sync · consent-first measurement · one contract, tested.